Bundesbrief

A confidence/truth layer over extracted Swiss parliamentary documents

Track: 2A — Academia Challenge (openparla / OpenParlData.ch) • **Team:** Powerworks — William Power • **Repository:** github.com/Powerworks-Global/bundesbrief • **License:** Apache 2.0

1. Summary

PDF-extraction pipelines for Swiss parliamentary data produce structured records, but have no insight into whether a given extraction was faithful to its source — a confident-looking record and a garbled one look identical once they are both just rows in a database. Bundesbrief wraps extraction in a **Trust Contract**: a per-record confidence score blending schema completeness, OCR/parse noise, source staleness, and — as of this submission — an independent **Apertus 8B cross-check** that re-reads the source text and judges how strongly it supports each extracted field. The score gates whether a record is auto-published (**Act**), flagged for human review (**Suggest**), or withheld entirely (**Halt**), logged to an append-only hash-chained ledger so every publish/withhold decision is independently verifiable after the fact.

Named for the 1291 Bundesbrief, the Swiss Confederation's founding charter: a confederation needed a document everyone could independently verify; an automated extraction pipeline needs the same thing for its own output.

2. The Problem

The openparla challenge asks for Swiss parliamentary affairs to be extracted from inconsistent PDF layouts into one structured, searchable format *while preserving links to the original source*. Bundesbrief satisfies the structural half of that brief (real PDF parsing, a source_url carried on every record) and treats the second, harder half — knowing when to trust the structured output — as the actual subject of the project. The gap is not hypothetical: in the recorded cross-check run over real Canton of Zurich PDFs (Section 5), the regex extractor emitted an affair_id that the independent Apertus check flatly rejected as unsupported by the source text — exactly the failure mode a trust layer exists to surface.

3. Architecture

Component	Role	Built this submission?
extraction.py	Real PDF field extraction via PyMuPDF (text-layer), named noise signals	Extended (was pre-extracted text)
apertus_crosscheck.py	Independent agreement signal: Apertus re-reads source text, scores each field	Yes — the submission
trust.py	Delegates scoring to the vendored trust_layer package; 4-dimension weighted blend	Extended (3 → 4 dimensions)
ledger.py	Append-only, SHA-256 hash-chained JSONL; /ledger/verify proves tamper-evidence	Pre-existing, untouched
api.py / mcp_server.py	FastAPI REST + MCP stdio server, same Trust Contract exposed both ways	Extended (new cross-check endpoint/tool)

```
raw PDF → extraction.py (PyMuPDF) → apertus_crosscheck.py (4th dimension, conditional) → trust.py (trust_layer scorer) → ledger.py (hash chain) → api.py / mcp_server.py
```

Each module is independently testable and unaware of the others' internals (ADR-001). Extraction and trust-scoring were deliberately decoupled before this submission; this decoupling is exactly what let the Apertus signal be added as a new, optional input rather than a redesign.

4. Use of Apertus

Model: a community Q4_K_M GGUF quantisation of swiss-ai/Apertus-v1.5-8B (repository Colby/apertus-v1.5-8b-text-Q4_K_M-GGUF, Apache-2.0, not gated), served locally via a local Ollama daemon — no hosted endpoint, no credential, no cloud VM. The exact pull form that resolves is ollama pull hf.co/Colby/apertus-v1.5-8b-text-Q4_K_M-GGUF:Q4_K_M (the plain repository-name form fails with a manifest error).

How it is used: as a cross-check, not an extraction engine. Apertus is given the raw source text and the regex-extracted field values, and returns a structured {agreement_score, discrepancies} judgement of how strongly the text supports each field. This becomes a fourth, independent dimension of the Trust Contract — the deterministic regex/PyMuPDF pipeline stays fully authoritative and auditable with no model in the loop; the model's opinion is additive, never a replacement.

Why conditional, not always-on: the existing hash-chained ledger had to stay backward compatible — ADR-002 had already demonstrated that one added field retroactively invalidates every pre-existing hash in a chain. The fourth

dimension is therefore active *only when a real measurement is attached*; an unmeasured, stub, or not-run record serialises identically to pre-submission Bundesbrief, byte for byte. This is proven empirically, not just reasoned: the repository's real pre-existing ledger files, and a 300-entry chain built with the pre-change code, all verify True under the new code; a pinned pre-change fixture chain is a permanent test. Full design record: ADR-003.

Weighting: when a real measurement exists, the weights rebalance from 0.50 / 0.30 / 0.20 (completeness / noise / freshness) to 0.40 / 0.24 / 0.16 / 0.20, keeping the deterministic trio's internal ratio and giving the model a bounded say: full disagreement (agreement 0.0) caps an otherwise-perfect record at 0.8 confidence — below the Act threshold, forcing Suggest — but a model opinion can never alone force a Halt. This split is reasoned, not tuned against a labelled dataset; ADR-003 states the explicit falsifier that would change it.

5. Empirical Cross-Check Run

Backend actually used: ollama:hf.co/Colby/apertus-v1.5-8b-text-Q4_K_M-GGUF:Q4_K_M — Upstream model: swiss-ai/Apertus-v1.5-8B. Weights actually run: community Q4_K_M GGUF quantisation served locally via Ollama (hf.co/Colby/apertus-v1.5-8b-text-Q4_K_M-GGUF:Q4_K_M). Numbers are from the quantised local weights, not the full upstream weights.
2 CPU cores, ~6.8 GB free memory, no accelerator; run sized to 3 samples + a few real Zurich documents EXECUTION LOCATION: this run was executed locally on the project workstation (2 CPU cores, no accelerator, quantised weights served by a local Ollama daemon, model dir under /tmp). NO cloud virtual machine was created for it, and none was touched during setup — which is why there is no instance identity or deletion evidence to show; the machine this artifact describes is the machine it ran on.

Document	Fields	Agreement	Conf. (determ.)	Conf. (w/ x-check)	Mode
GR-2026-0412	fixture	1.00	0.9905	0.9905	ACT
GR-2026-0413	fixture	1.00	0.7775	0.7775	SUGGEST
GR-1998-0071	fixture	1.00	0.4850	0.4850	HALT
VNL1002006	0	n/a	n/a	n/a	not extracted - nothing to cross-check
VNL1002005	1/5	0.00	0.5898	0.4718	HALT
VNL1001999	0	n/a	n/a	n/a	not extracted - nothing to cross-check
VNL1002000	1/5	0.00	0.5916	0.4733	HALT

The two halting real Zurich documents are the point of the exercise: VNL1002005 (1 of 5 expected fields found) and VNL1002000 scored 0.59 and 0.59 on the deterministic trio alone — inside the auto-publish band on completeness arithmetic — but the independent Apertus re-read returned agreement 0.0 (discrepancy: the extracted affair id is unsupported by the source text), pulling both under the Act threshold to **HALT**. A confident-looking row from a 20%-complete extraction is exactly the failure the Trust Contract exists to catch.

Ledger after the run: {"valid": true, "entries": 3} across 3 entries (append-only, SHA-256 hash-chained). The deterministic test suite passes **100 tests** in 0.393s, including end-to-end inversion properties: a noisier PDF must score strictly lower confidence, and a noise dimension scores correctly only when registered as inverted.

Note on backends: unit-test fixtures and harness validation run use an explicit *stub* cross-check backend (labelled stub:substring-presence, never presented as a model measurement); every number above comes from the real-model artifact committed as data/-grade provenance alongside this report.

6. Limitations & Reproduce

Honest limits, stated before someone finds them: (1) numbers are from the community **Q4_K_M quantisation** on 2 CPU cores, not the full-precision upstream weights; (2) the empirical sample is small — three fixtures plus four real Zurich PDFs — chosen to demonstrate all three modes, not to estimate precision; (3) the 0.40/0.24/0.16/0.20 weighting is reasoned, not tuned against a labelled dataset (ADR-003 names the falsifier); (4) extraction targets the text layer — scanned-image PDFs need OCR before this pipeline adds value.

```
git clone https://github.com/Powerworks-Global/bundesbrief → pip install -e . → ollama pull hf.co/Colby/apertus-v1.5-8b-text-Q4_K_M-GGUF:Q4_K_M → BUNDESBRIEF_REAL_BACKENDS=1 make demo → make test → curl /ledger/verify
```

Apache-2.0 — fork, audit, reuse freely.